P3798R0
The unexpected in std::expected

Published Proposal, 2025-06-29

This version:

https://godexsoft.github.io/papers/P3798R0.html

Authors:

Alex Kremer (Ripple)

Ayaz Salikhov (Ripple)

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

https://github.com/godexsoft/papers/blob/master/source/P3798R0.bs

---

Abstract

We propose adding a `has_error()` member function to `std::expected` to complement the existing `has_value()` functionality.

Table of Contents

§ 1. Motivation

Today, `std::expected` provides a `has_value()` member function that can be used to check whether the instance holds a value or an error. There is also an implicit conversion operator to `bool` that can be used for the same purpose. These two existing mechanisms follow several other facilities in the language, including `std::optional`.

While `std::optional` provides only the `has_value()` member function as its primary state-checking mechanism, `std::expected` serves a fundamentally different purpose. Unlike `std::optional`, which represents either a value or nothing, `std::expected` represents either a value or an error. This semantic difference warrants distinct interface considerations. Adding `has_error()` creates symmetry in the API that better reflects the dual-state nature of `std::expected` and provides more readable, self-documenting code when the focus is on error handling rather than value presence.

§ 1.1. Sample usecase

Consider the following examples (assuming `result` is of type `std::expected<int, std::string>`):

Without this proposal	With this proposal
`if (!result.has_value())` `  log_and_exit(result.error());`	`if (result.has_error())` `  log_and_exit(result.error());`

`// somewhere in unit tests`
`ASSERT_TRUE(!result.has_value());`
`EXPECT_EQ(result.error(), "myError");`
	`// somewhere in unit tests`
	`ASSERT_TRUE(result.has_error());`
	`EXPECT_EQ(result.error(), "myError");`

§ 1.2. Impact on the standard

This change is entirely based on library extensions and does not require any language features beyond what is available in C++ 23.

§ 1.3. Other languages

It might be unexpected to not have a `has_error()` method in `std::expected`, especially when moving from other languages that have similar constructs.

The proposed approach is consistent with similar facilities in some other programming languages:

- **Rust** provides both `is_ok()` and `is_err()` methods on its `Result` type.

- **D** provides both `hasValue` and `hasError` properties in its `expected` package.
- **Haskell** provides `isRight` and `isLeft` in its `Either` type.

These implementations acknowledge the dual-state nature of error-handling types by offering explicit methods for checking both states, rather than relying solely on negation of a value-checking method.

§ 2. Proposed Wording

In 22.8.6.1 [**expected.object.general**] add:

```
constexpr bool has_value() const noexcept;
constexpr bool has_error() const noexcept;
constexpr const T& value() const &;
```

In 22.8.6.6 [**expected.object.obs**] add after ¶7:

```
constexpr bool has_error() const noexcept;
    Returns: !has_val.
    Remarks: Equivalent to !has_value()
```

In 22.8.7.1 [**expected.void.general**] add:

```
constexpr bool has_value() const noexcept;
constexpr bool has_error() const noexcept;
constexpr void operator*() const noexcept;
```

In 22.8.7.6 [**expected.void.obs**] add after ¶1:

```
constexpr bool has_error() const noexcept;
    Returns: !has_val.
    Remarks: Equivalent to !has_value()
```

§ References

§ Informative References

[P0032R2]

Vicente J. Botet Escriba. *Homogeneous interface for variant, any and optional (Revision 2)*. 13 March 2016. URL: https://wg21.link/p0032r2

[P0323R12]

Vicente Botet, JF Bastien, Jonathan Wakely. *std::expected*. 7 January 2022. URL: https://wg21.link/p0323r12